

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA
MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

CORSO DI STUDIO IN
Informatica

TESI DI LAUREA IN
Programmazione Web

TITOLO

*Studio e realizzazione di soluzioni basate su crittografia omomorfica per
garantire la privacy delle applicazioni medicali*

Relatore:

Chiar.mo Prof.
Pierpaolo Loreti

Laureando:

matricola: 0336760
Doru Ionut Parvu

Correlatrice:

Dott. Ing.
Giorgia Panico

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Obiettivo della tesi	2
1.3	Percorso seguito	2
1.4	Contributi principali	3
2	Tecniche di cifratura omomorfica	4
2.1	Calcolare senza decifrare	4
2.2	Base tecnologica: reticoli e rumore	5
2.2.1	Leveled FHE e bootstrapping	5
2.3	Dati inseriti negli slot	5
2.3.1	Dimensione dell'anello e numero di slot	6
2.3.2	Rotazioni	6
2.4	BFV e BGV: interi esatti modulo un primo	6
2.4.1	Differenza pratica tra BFV e BGV	6
2.5	CKKS: numeri reali e complessi approssimati	7
2.6	Parametri che cambiano il costo	7
2.6.1	Profondità moltiplicativa	7
2.6.2	Modulo del plaintext e valori massimi	8
2.6.3	Batch e slot utilizzati	8
2.7	Limiti distinti	8
2.7.1	Limiti degli schemi	8
2.7.2	Limiti della libreria	8
2.8	Scelta dello schema	9
3	Benchmark degli schemi omomorfici	10
3.1	Obiettivo del confronto	10
3.2	Ambiente sperimentale	10
3.2.1	Parametri variati	11
3.2.2	Avvertenza sulla sicurezza	11
3.3	Operazioni misurate	11

3.4	Somma e moltiplicazione	12
3.5	Memoria e interpretazione dei risultati	14
3.6	Il costo particolare del bootstrapping	14
3.7	Scelta dello schema	15
3.8	Limiti del benchmark	16
3.9	Conclusioni	16
4	Applicazioni di elaborazioni omomorfe a scenari medicali	17
4.1	Introduzione	17
4.2	Richiami sulla regressione lineare	18
4.2.1	Funzione di costo	18
4.2.2	Determinazione dei coefficienti	19
4.2.3	Interpretazione in ambito medico	20
4.3	Regressione lineare ed elaborazione omomorfa	20
4.4	Scenario 1: query cifrata al computation server	21
4.4.1	Schema logico	22
4.4.2	Distribuzione delle chiavi	23
4.5	Scenario 2: il polinomio cifrato viene trasferito al decryptor	24
4.5.1	Schema logico	24
4.5.2	Distribuzione delle chiavi	25
4.6	Confronto tra le due architetture	26
4.7	Esempio applicativo: stima dei giorni di ricovero	27
4.7.1	Come parte il training	27
4.7.2	Quale modello viene recuperato	27
4.7.3	Verifica su un nuovo dato	27
4.7.4	Lettura dell'esempio nei due scenari	28
4.8	Aspetti operativi e limiti	28
4.9	Conclusioni	28
5	Protocollo di aggregazione simil prio3 utilizzando tecniche omomorfe	29
5.1	Introduzione	29
5.2	Idea generale di fhe-vdaf	29
5.3	Ruoli architetturali e gestione delle chiavi	30
5.3.1	Attori del sistema	31
5.3.2	Chi possiede quali chiavi	31
5.3.3	Conseguenze di sicurezza	31
5.4	Architettura complessiva in uno scenario di dati aggregati	32
5.4.1	Schema dei messaggi	32
5.5	Conversione dell'input in binario	32
5.5.1	Perché usare una rappresentazione binaria	32

5.5.2	Vincolo sul valore massimo	32
5.5.3	Esempi di codifica	34
5.6	Validazione omomorfica dell'input	34
5.6.1	Controllo che ogni slot sia un bit	34
5.6.2	Da errore a flag di validità	35
5.6.3	Validate-or-zero	36
5.7	Aggregazione dei report validi	36
5.8	Esempio completo su dati aggregati	37
5.8.1	Codifica dei valori	37
5.8.2	Somma per slot	37
5.8.3	Decodifica finale	37
5.9	Punti di forza e limiti architetturali	38
5.9.1	Vantaggi	38
5.9.2	Limiti	38
5.10	Conclusioni	38
6	Conclusioni	40

Capitolo 1

Introduzione

1.1 Contesto e motivazione

La quantità di dati raccolti da organizzazioni pubbliche e private cresce continuamente. Ospedali, centri di ricerca, aziende e servizi digitali producono informazioni che possono essere utili per costruire modelli statistici, calcolare indicatori o conoscere l'andamento di un fenomeno. Molti di questi dati sono però sensibili. Un dato medico, una misura economica o una preferenza personale non dovrebbe essere consegnata in chiaro a ogni soggetto che partecipa all'elaborazione.

La soluzione più immediata consiste nel cifrare i dati quando vengono salvati o trasmessi. La cifratura tradizionale protegge bene queste due fasi, ma normalmente richiede di decifrare le informazioni prima di elaborarle. Il server che esegue il calcolo torna quindi a vedere i valori originali. Questo passaggio crea un punto delicato: per ottenere un risultato bisogna concedere fiducia all'infrastruttura che esegue il calcolo.

La cifratura omomorfica affronta il problema da una prospettiva diversa. Essa permette di eseguire alcune operazioni direttamente sui dati cifrati. Il server riceve ciphertext, cioè rappresentazioni non leggibili, applica somme o moltiplicazioni e produce un nuovo ciphertext. Dopo la decifratura, il risultato corrisponde al calcolo che sarebbe stato eseguito sui dati in chiaro. Il server può quindi contribuire all'elaborazione senza conoscere necessariamente gli input.

Questa proprietà è interessante, ma non rende il calcolo cifrato gratuito o automaticamente adatto a ogni applicazione. Le operazioni omomorfe sono più lente delle operazioni ordinarie, richiedono più memoria e dipendono da parametri che devono essere scelti con attenzione. Schemi diversi rappresentano tipi di dato diversi e offrono compromessi differenti tra precisione, capacità di calcolo e costo.

1.2 Obiettivo della tesi

L'obiettivo di questa tesi è studiare in modo progressivo come le tecniche omomorfiche possano essere usate per elaborare e aggregare dati proteggendo maggiormente i contributi individuali. Il lavoro non si limita a descrivere gli schemi da un punto di vista teorico. La parte centrale consiste nel collegare le loro caratteristiche a misure sperimentali e, successivamente, a due applicazioni concrete.

Le domande che guidano il lavoro sono semplici:

- quali tipi di dato possono essere rappresentati dai principali schemi omomorfici;
- quali parametri incidono maggiormente sul costo delle operazioni;
- quale differenza pratica esiste tra sommare e moltiplicare dati cifrati;
- come usare queste tecniche in un modello di regressione lineare;
- come costruire un protocollo di aggregazione ispirato alle VDAF, nel quale il risultato collettivo possa essere recuperato senza trattare in chiaro ogni contributo.

La tesi considera tre schemi disponibili nella libreria OpenFHE: BFV, BGV e CKKS. BFV e BGV sono adatti a operazioni esatte su interi rappresentati in uno spazio modulare. CKKS è invece progettato per calcoli approssimati su valori reali o complessi ed è quindi utile per molte elaborazioni statistiche e di machine learning [1, 2].

1.3 Percorso seguito

Il lavoro è iniziato dallo studio tecnologico degli schemi omomorfici. Prima di scegliere uno schema per un'applicazione, è infatti necessario capire che cosa possa rappresentare, quali operazioni supporti e come cambino i ciphertext durante il calcolo. Particolare attenzione è stata dedicata alla dimensione dell'anello, al numero di slot disponibili, al modulo del plaintext e alla profondità moltiplicativa.

Il secondo passo è stato la costruzione di una suite di benchmark basata su OpenFHE. Gli esperimenti misurano il tempo di creazione del contesto, la generazione delle chiavi, la cifratura, la decifratura, la somma e la moltiplicazione omomorfe. Per CKKS sono stati misurati anche la generazione delle chiavi di bootstrapping e il bootstrapping stesso. I test variano la dimensione dell'anello e la profondità richiesta, in modo da osservare come aumentino tempo e memoria.

I benchmark non servono a dichiarare uno schema vincitore in ogni situazione. Il loro scopo è mostrare che la scelta dipende dal problema. Se i dati sono conteggi interi e si desiderano risultati esatti, BFV o BGV sono candidati naturali. Se si elaborano medie,

coefficienti o misure non intere, CKKS offre una rappresentazione più adatta, accettando un piccolo errore numerico.

Dopo il confronto sperimentale, il lavoro applica CKKS a un caso di machine learning semplice e interpretabile: la regressione lineare. Il caso di studio usa dati sanitari e stima la durata di un ricovero. La regressione è stata scelta perché permette di osservare con chiarezza il passaggio dai dati cifrati al modello e perché l'inferenza richiede principalmente somme e moltiplicazioni.

L'ultima parte si sposta dal machine learning all'aggregazione verificabile. Viene presentato un protocollo ispirato a VDAF Prio3, ma reinterpretato tramite BGV. I client cifrano report interi, l'aggregatore verifica sotto cifratura che la loro rappresentazione sia valida e somma i contributi accettati. Il collector decifra il totale finale. L'obiettivo non è riprodurre integralmente Prio3, ma studiare una costruzione più semplice che mantenga l'idea di apprendere un aggregato senza esporre direttamente ogni report.

1.4 Contributi principali

I contributi del lavoro possono essere riassunti in quattro punti.

- (a) Una descrizione accessibile delle caratteristiche operative di BFV, BGV e CKKS, con una distinzione tra limiti dello schema, limiti della libreria e limiti della configurazione scelta.
- (b) Una suite sperimentale comune per confrontare le principali operazioni omomorfe al variare della dimensione dell'anello e della profondità moltiplicativa.
- (c) Un'applicazione di CKKS alla regressione lineare in uno scenario sanitario, con due possibili distribuzioni dei ruoli di calcolo e decifratura.
- (d) Una costruzione di aggregazione VDAF-like basata su BGV, con codifica binaria, validazione omomorfa e decifratura del solo risultato aggregato.

Il filo comune è il passaggio dalla singola operazione crittografica a un sistema completo. La scelta dello schema influenza il formato dei dati; i parametri influenzano costo e capacità; l'architettura stabilisce quali attori possano cifrare, calcolare e decifrare. La privacy non dipende quindi da un solo algoritmo, ma dall'insieme di queste decisioni.

Capitolo 2

Tecniche di cifratura omomorfica

2.1 Calcolare senza decifrare

In una cifratura tradizionale un mittente trasforma un messaggio leggibile in un ciphertext usando una chiave. Chi possiede la chiave corretta può recuperare il messaggio originale. Se il dato deve essere elaborato, normalmente viene prima decifrato.

La cifratura omomorfica aggiunge una possibilità: alcune operazioni possono essere eseguite direttamente sui ciphertext. Si può immaginare un'azienda che voglia conoscere la somma delle vendite di più negozi senza leggere il valore di ciascun negozio. Ogni sede cifra il proprio dato; il server somma i ciphertext; il soggetto autorizzato decifra soltanto il totale.

Un sistema di questo tipo comprende normalmente:

- una fase di configurazione, nella quale vengono scelti schema e parametri;
- la generazione di una chiave pubblica e di una chiave segreta;
- eventuali chiavi di valutazione, necessarie per operazioni come moltiplicazioni e rotazioni;
- la codifica e la cifratura dei dati;
- la valutazione omomorfica;
- la decifratura e la decodifica del risultato.

La chiave pubblica può essere distribuita ai soggetti che devono cifrare. La chiave segreta deve invece restare presso il soggetto autorizzato a leggere il risultato. Le chiavi di valutazione consentono al server di trasformare i ciphertext durante il calcolo, ma non dovrebbero permettergli di recuperarne il contenuto.

2.2 Base tecnologica: reticoli e rumore

Gli schemi considerati in questa tesi si basano su problemi matematici legati ai reticoli. Per comprendere il funzionamento operativo non è necessario studiare la geometria completa di questi oggetti. È sufficiente osservare che un ciphertext contiene il messaggio in una forma nascosta insieme a una componente casuale, spesso chiamata rumore.

Il rumore è utile per la sicurezza, perché impedisce di riconoscere direttamente il messaggio. Durante le operazioni omomorfe, tuttavia, anche il rumore cambia. Le somme lo aumentano in modo generalmente contenuto; le moltiplicazioni hanno un effetto molto maggiore. Se il rumore supera il margine previsto dai parametri, la decifratura può restituire un valore errato.

Per questo motivo il programma da eseguire deve essere considerato prima di creare il contesto crittografico. Non basta sapere quanti dati saranno cifrati: bisogna conoscere soprattutto quante moltiplicazioni consecutive saranno necessarie. Questo valore è chiamato profondità moltiplicativa.

2.2.1 Leveled FHE e bootstrapping

Un sistema *leveled* viene configurato per sostenere una profondità massima. Se il calcolo resta entro quel limite, non è necessario rinnovare il ciphertext. BGV ha reso particolarmente importante questa impostazione, mostrando come eseguire circuiti di profondità prefissata senza ricorrere obbligatoriamente al bootstrapping [3].

Il bootstrapping è un'operazione più complessa che riduce l'effetto del rumore e rende nuovamente utilizzabile un ciphertext vicino al proprio limite. Permette di proseguire calcoli più lunghi, ma richiede chiavi aggiuntive, memoria e molto più tempo rispetto a una somma o a una singola moltiplicazione. In pratica si preferisce evitarlo quando il calcolo può essere dimensionato in anticipo.

2.3 Dati inseriti negli slot

I ciphertext moderni non devono necessariamente contenere un solo numero. Attraverso il *packing*, più valori vengono inseriti in posizioni chiamate slot. Una singola operazione può quindi agire contemporaneamente su molti elementi, secondo un modello simile al SIMD: la stessa istruzione viene applicata in parallelo a più dati.

Se due ciphertext contengono rispettivamente i vettori

$$[2, 4, 6, 8] \quad \text{e} \quad [1, 3, 5, 7],$$

la loro somma omomorfica produce un ciphertext che, una volta decifrato, contiene

$$[3, 7, 11, 15].$$

Il costo dell'operazione non cresce come se fossero state eseguite quattro cifrature separate. Il packing è quindi essenziale per rendere pratiche aggregazioni e operazioni vettoriali.

2.3.1 Dimensione dell'anello e numero di slot

La *ring dimension*, indicata spesso con N , è uno dei parametri centrali. Una dimensione maggiore può offrire più slot e sostenere parametri crittografici più grandi, ma aumenta anche la dimensione dei ciphertext, il tempo delle operazioni e il consumo di memoria.

Nell'implementazione di benchmark usata in questa tesi, BFV e BGV impiegano fino a N slot con il packed encoding, mentre CKKS utilizza $N/2$ slot complessi.

Una dimensione maggiore non significa automaticamente maggiore efficienza. Se l'applicazione usa pochi valori, gran parte degli slot rimane vuota mentre il costo del ciphertext resta elevato. Il vantaggio emerge quando gli slot vengono riempiti con un batch abbastanza grande.

2.3.2 Rotazioni

Le somme e le moltiplicazioni agiscono slot per slot. Per combinare elementi che occupano posizioni diverse servono le rotazioni. Una rotazione sposta ciclicamente il contenuto degli slot; ripetendo rotazioni e somme si può, ad esempio, calcolare la somma di tutti gli elementi di un vettore.

Le rotazioni richiedono evaluation keys dedicate. Generare e conservare molte chiavi di rotazione può avere un costo rilevante. Una buona disposizione dei dati negli slot riduce il numero di rotazioni richieste e può incidere sull'efficienza quanto la scelta dello schema.

2.4 BFV e BGV: interi esatti modulo un primo

BFV, derivato dal lavoro di Fan e Vercauteren [4], e BGV [3] rappresentano dati interi. Questi valori appartengono a uno spazio finito determinato dal *plaintext modulus*, indicato con p . In pratica, p stabilisce quali valori possono essere rappresentati direttamente: ogni somma o moltiplicazione viene ricondotta a un risultato compreso tra 0 e $p - 1$.

2.4.1 Differenza pratica tra BFV e BGV

BFV e BGV condividono molte caratteristiche visibili all'utilizzatore: dati interi, risultati esatti entro lo spazio modulare, packing e operazioni aritmetiche. Differiscono

nella costruzione interna e nella gestione del rumore. Di conseguenza, a parità apparente di parametri, possono mostrare tempi e consumi diversi.

Occorre verificare il circuito reale, i parametri supportati dalla libreria e le prestazioni sulla macchina destinata all'esecuzione. Nel protocollo di aggregazione di questa tesi viene usato BGV perché i report sono interi e la validazione sfrutta direttamente proprietà dell'aritmetica modulo p . Infine un grosso vantaggio dello schema BGV è la velocità nell'eseguire operazioni di moltiplicazioni relativo a BFV.

2.5 CKKS: numeri reali e complessi approssimati

CKKS nasce per elaborare numeri reali o complessi in forma approssimata [2]. Questa caratteristica lo rende adatto a regressione, statistica e machine learning, dove coefficienti e risultati intermedi non sono necessariamente interi.

Il valore viene codificato usando una scala. Dopo alcune operazioni, in particolare le moltiplicazioni, la scala e la dimensione numerica dei coefficienti devono essere gestite attraverso il *rescaling*. Il risultato decifrato non coincide sempre cifra per cifra con il calcolo in chiaro, ma dovrebbe essere vicino entro la precisione prevista.

Per esempio, un risultato teorico pari a 12,5 potrebbe essere decifrato come 12,50001. In un modello statistico tale differenza può essere trascurabile; in un conteggio esatto o in una logica che richiede uguaglianze perfette potrebbe invece essere inaccettabile.

CKKS può rappresentare anche valori complessi. Quando l'applicazione usa soltanto numeri reali, la struttura complessa resta parte del meccanismo di packing. In modo intuitivo si può dire che ogni slot ospita un valore complesso, formato da una parte reale e da una parte immaginaria; per questo, nella configurazione adottata, gli slot indipendenti disponibili sono normalmente pari a $N/2$.

2.6 Parametri che cambiano il costo

2.6.1 Profondità moltiplicativa

La profondità non è il numero totale di moltiplicazioni, ma il massimo numero di moltiplicazioni lungo una sequenza dipendente. Dieci prodotti indipendenti possono essere eseguiti allo stesso livello; il prodotto del risultato precedente per un nuovo valore consuma invece un altro livello.

Richiedere una profondità molto superiore al necessario produce contesti, chiavi e ciphertext più costosi. Richiederne troppo poca impedisce di completare il calcolo. La prima ottimizzazione consiste quindi nel descrivere il circuito e nel ridurre le moltiplicazioni consecutive.

2.6.2 Modulo del plaintext e valori massimi

Per BFV e BGV il modulo del plaintext stabilisce lo spazio degli interi rappresentabili e influenza il batching. Un modulo più grande consente risultati intermedi più ampi, ma interagisce con gli altri parametri e non può essere aumentato senza conseguenze.

Nel benchmark BFV e BGV usano il primo 786433. La scelta permette di confrontare i due schemi con lo stesso spazio del plaintext e supporta il packing per le dimensioni considerate. Non significa che questo primo sia adatto a ogni applicazione.

2.6.3 Batch e slot utilizzati

Il *batch length* è il numero di valori effettivamente inseriti nel ciphertext. Può essere inferiore alla capacità massima. Usare più slot aumenta il lavoro utile svolto da una singola operazione, ma richiede una disposizione coerente dei dati.

In uno scenario con migliaia di contributi, il server può aggregare più valori per ciphertext. In uno scenario con un solo valore, la stessa configurazione può risultare inefficiente. Per questo i benchmark devono essere letti insieme al numero di elementi elaborati e non soltanto come latenza della singola operazione.

2.7 Limiti distinti

2.7.1 Limiti degli schemi

Gli schemi impongono vincoli matematici: crescita del rumore, profondità limitata senza bootstrapping, aritmetica modulare per BFV e BGV, approssimazione per CKKS. Inoltre, solo le operazioni esprimibili tramite il circuito supportato possono essere valutate direttamente. Confronti, divisioni e funzioni non lineari possono richiedere approssimazioni, costruzioni più costose oppure non essere praticabili direttamente.

2.7.2 Limiti della libreria

OpenFHE offre implementazioni, generatori di contesto, packing e rotazioni per più schemi [1]. Il bootstrapping, nel contesto considerato in questa tesi, viene invece usato per CKKS. La libreria riduce la complessità di sviluppo, ma non sceglie automaticamente la migliore architettura applicativa. Versione, API disponibili, tecniche di scaling e operazioni implementate determinano ciò che il programma può usare concretamente.

Anche la serializzazione e la distribuzione delle evaluation keys sono responsabilità del sistema.

2.8 Scelta dello schema

Una regola pratica iniziale può essere formulata così:

- BFV o BGV quando gli input sono interi, il risultato deve essere esatto e l'aritmetica modulare è gestita esplicitamente;
- CKKS quando gli input includono valori reali o complessi e una piccola approssimazione è accettabile;

Capitolo 3

Benchmark degli schemi omomorfici

3.1 Obiettivo del confronto

Il capitolo precedente ha mostrato che uno schema omomorfico non viene scelto soltanto in base al livello di privacy desiderato. Tipo di dato, profondità del calcolo, numero di slot e memoria disponibile influenzano la soluzione. I benchmark servono a rendere visibile tale costo.

La suite sperimentale confronta BFV, BGV e CKKS nella libreria OpenFHE. Il capitolo si concentra sui microbenchmark delle principali operazioni, in modo da mostrare come cambino tempo e memoria al variare dei parametri.

L'obiettivo non è stabilire una classifica assoluta. BFV e BGV lavorano su interi esatti modulo p , mentre CKKS lavora su numeri approssimati. Un tempo inferiore non rende uno schema adatto a un tipo di dato che non rappresenta correttamente.

3.2 Ambiente sperimentale

I benchmark sono scritti in C++17 e usano OpenFHE insieme a Google Benchmark. Il codice della suite sperimentale è disponibile in una repository dedicata [5]. I risultati sono esportati in formato CSV e successivamente elaborati con Python. La macchina di prova ha le seguenti caratteristiche:

- CPU AMD Ryzen 9800X3D, con otto core rilevati dal benchmark;
- 48 GB di memoria DDR5;
- ambiente Windows Subsystem for Linux;
- compilazione ottimizzata con `-march=native`;
- uso di OpenMP nelle operazioni che supportano il parallelismo.

La creazione del contesto viene eseguita su un singolo core. Le altre operazioni possono sfruttare più thread. Il tempo riportato nei grafici è il tempo CPU medio prodotto da Google Benchmark. La memoria è una stima dell'heap osservato tramite l'allocatore di sistema; non include lo swap e deve essere interpretata come indicazione relativa.

3.2.1 Parametri variati

La suite varia due parametri:

- la profondità moltiplicativa, da circuiti molto brevi a configurazioni molto profonde;
- la ring dimension, da 256 fino a 16384 per il confronto comune, con valori superiori disponibili per BGV e CKKS.

BFV e BGV usano il plaintext modulus primo $p = 786433$. CKKS usa una configurazione di scaling dedicata ai valori approssimati. Per le figure principali viene fissata una profondità pari a 16, perché consente di confrontare più schemi sulla stessa base e rappresenta un buon compromesso per molti scenari applicativi: non è troppo bassa, quindi permette calcoli non banali, ma non è nemmeno così alta da rendere il confronto poco leggibile e le operazioni troppo dispendiose.

3.2.2 Avvertenza sulla sicurezza

Nel codice dei tre benchmark il livello di sicurezza è impostato a `HEStd_NotSet`. La ring dimension viene quindi forzata per studiarne l'effetto sulle prestazioni. Questa impostazione è utile per un esperimento controllato, ma non garantisce che ogni coppia di parametri raggiunga un livello di sicurezza standard.

I grafici non devono essere letti come raccomandazioni di configurazione. In un sistema reale il livello di sicurezza deve essere fissato e i parametri devono essere validati secondo le indicazioni della libreria e degli standard applicabili.

3.3 Operazioni misurate

La suite misura:

- `ContextCreation`, cioè la costruzione dell'ambiente crittografico;
- `KeyGen`, per chiave pubblica e segreta;
- `EvalKeyGen`, per chiavi di moltiplicazione e rotazione;
- `Encrypt`, che cifra i dati con la chiave pubblica;
- `Decrypt`, che recupera il risultato con la chiave segreta;

- **EvalAdd**, che somma due ciphertext mantenendo il risultato cifrato;
- **EvalMult**, che moltiplica due ciphertext e consuma un livello;
- per CKKS, **BootstrapKeyGen**, che genera le chiavi necessarie al bootstrapping;
- per CKKS, **Bootstrap**, che rinnova un ciphertext per continuare il calcolo.

In forma intuitiva, se **Encrypt** produce due ciphertext che proteggono i valori a e b , **EvalAdd** costruisce la cifratura di $a + b$, mentre **EvalMult** costruisce la cifratura di $a \cdot b$. Solo **Decrypt** rende leggibile il risultato finale.

Le misure sono microbenchmark: ogni test isola una singola operazione. Non includono trasferimento in rete, serializzazione, lettura dei dati o coordinamento tra attori. Sono utili per capire dove si concentra il costo, ma non equivalgono alla latenza completa di un'applicazione.

3.4 Somma e moltiplicazione

Le Figure 3.1 e 3.2 confrontano le due principali operazioni eseguite sui ciphertext.

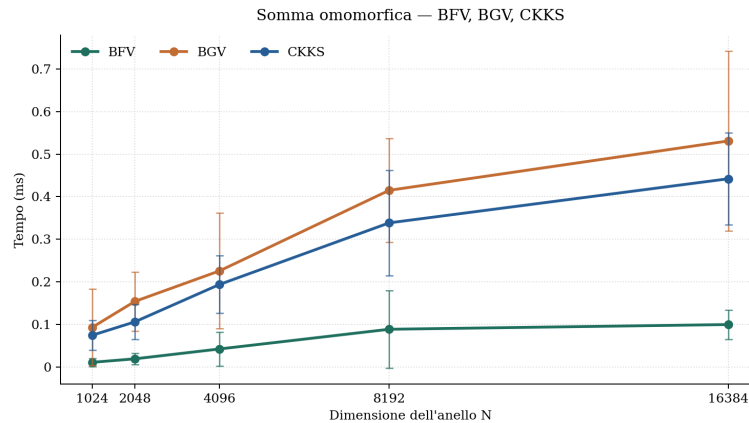


Figura 3.1: Tempo di una somma omomorfica con profondità 16.

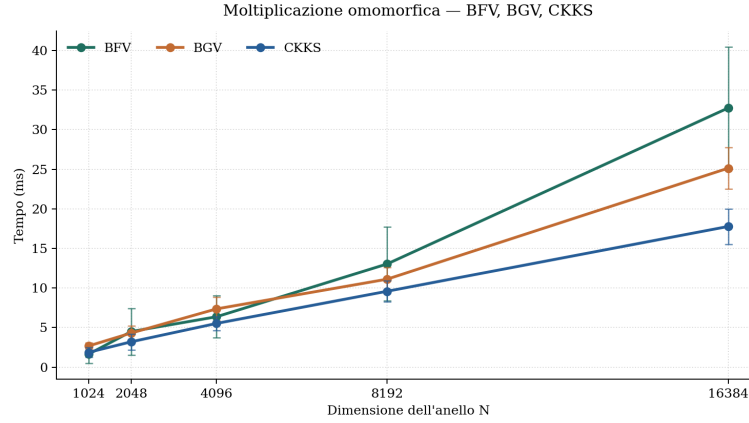


Figura 3.2: Tempo di una moltiplicazione omomorfica con profondità 16.

La somma è chiaramente l'operazione più economica. I tre schemi restano tutti su valori molto bassi, con differenze contenute, anche se BGV tende a risultare un po' più pesante degli altri due. CKKS mostra l'andamento più leggero nel complesso, mentre BFV rimane vicino soprattutto nelle dimensioni intermedie e grandi.

La moltiplicazione cambia nettamente il quadro. Tutti gli schemi crescono con la ring dimension, ma BFV diventa molto più costoso di BGV e CKKS lungo quasi tutto l'intervallo osservato. Tra questi ultimi due, CKKS risulta in genere leggermente più rapido, mentre BGV gli resta vicino. Il dato principale che emerge dal grafico è quindi la distanza tra `EvalAdd` e `EvalMult`, e soprattutto il peso molto maggiore della moltiplicazione in BFV.

Le somme beneficiano del packing, che applica la stessa operazione a tutti gli slot. Quando invece il calcolo richiede controlli, prodotti tra valori o passaggi di machine learning, le moltiplicazioni diventano una parte importante della latenza.

Una dimensione maggiore offre anche più slot: BFV e BGV possono inserire N interi, mentre CKKS inserisce $N/2$ valori complessi. Questo aumenta la quantità di dati elaborabili in parallelo, ma rende anche più costose le singole operazioni.

Operazione, $N = 4096$	BFV	BGV	CKKS
Cifratura	5,496 ms	3,906 ms	3,846 ms
Somma	0,158 ms	0,232 ms	0,085 ms
Moltiplicazione	13,652 ms	4,392 ms	3,879 ms
Decifratura	0,358 ms	2,849 ms	2,827 ms

Tabella 3.1: Tempi CPU con profondità 16 e ring dimension 4096.

La Tabella 3.1 mostra che una moltiplicazione costa molto più di una somma. Inoltre, le moltiplicazioni consumano livelli e richiedono evaluation keys. Il costo di una validazione omomorfica non dipende quindi solo dal numero di report, ma soprattutto dalla difficoltà del circuito in cui essi saranno valutati.

3.5 Memoria e interpretazione dei risultati

La memoria registrata dalla suite non rappresenta il costo isolato del singolo ciphertext. Il processo mantiene contesto, chiavi e strutture create per il benchmark corrente. Inoltre, l'allocatore può conservare memoria tra operazioni. I valori devono quindi essere letti come impronta del processo e non come dimensione esatta di un oggetto.

La Figura 3.3 mostra comunque la tendenza generale al crescere di N .

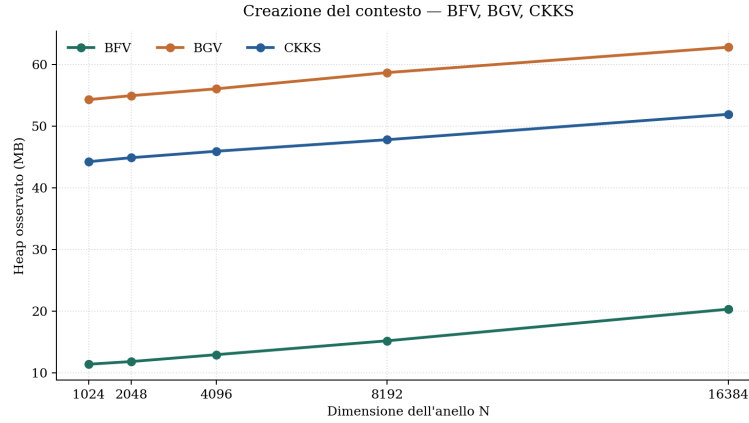


Figura 3.3: Heap osservato durante la creazione del contesto, con profondità 16.

BFV mostra nel processo di prova un livello di memoria molto inferiore rispetto a BGV e CKKS. I valori assoluti di BGV e CKKS includono memoria mantenuta dalla sequenza sperimentale e non devono essere interpretati come un confronto perfettamente isolato. La conclusione affidabile è più generale: contesti profondi, dimensioni maggiori ed evaluation keys possono richiedere gigabyte, rendendo la memoria un vincolo reale.

3.6 Il costo particolare del bootstrapping

CKKS include due benchmark aggiuntivi dedicati al bootstrapping. Il suo costo non dipende direttamente dal valore di profondità usato, ma soprattutto dalla ring dimension. In pratica, aumentando N , anche il costo del bootstrapping cresce in modo evidente.

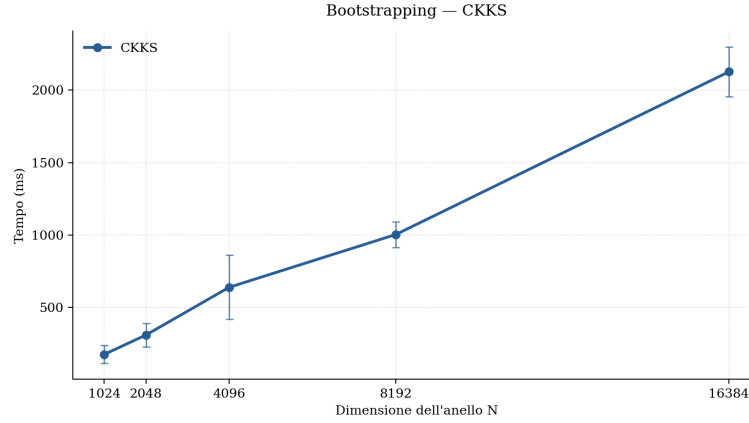


Figura 3.4: Tempo di bootstrapping in CKKS con profondità 16.

Questo rende il bootstrapping un'operazione molto più pesante delle somme e delle moltiplicazioni ordinarie. In calcoli poco profondi non è quindi conveniente, mentre diventa interessante soltanto quando serve proseguire un'elaborazione che altrimenti esaurirebbe i livelli disponibili.

Il confronto evidenzia una regola pratica: prima di ricorrere al bootstrapping conviene aumentare la profondità, separare il calcolo in fasi o verificare se una configurazione leveled sia sufficiente.

3.7 Scelta dello schema

Se si vogliono usare numeri interi oppure aritmetica modulare, la scelta naturale è BFV o BGV. Tra questi due, BFV può andare bene quando il calcolo resta semplice, mentre BGV diventa in genere preferibile quando si usano operazioni più dispendiose della sola somma, in particolare le moltiplicazioni.

CKKS è invece la scelta adatta quando i dati non sono interi oppure quando si vogliono rappresentare numeri complessi. In questo caso bisogna accettare che il risultato sia approssimato e non esatto come negli schemi basati su interi.

	BFV	BGV	CKKS
Dati	Interi	Interi	Reali o complessi
Risultato	Esatto modulo p	Esatto modulo p	Approssimato
Caso naturale	Conteggi e somme	Conteggi e controlli modulari	Statistica e machine learning
Attenzione	Overflow modulare	Overflow e profondità	Precisione e rescaling

Tabella 3.2: Lettura applicativa dei tre schemi.

3.8 Limiti del benchmark

I risultati sono stati raccolti su una sola macchina e in un ambiente WSL. Non sono presenti intervalli di confidenza tra esecuzioni indipendenti, misure energetiche o costi di rete. La memoria dipende dal comportamento dell’allocatore e dall’ordine dei test.

Il confronto usa parametri simili per rendere visibili le tendenze, ma gli schemi non offrono esattamente la stessa semantica. Infine, l’uso di `HEStd_NotSet` impedisce di associare automaticamente ogni punto a uno specifico livello di sicurezza.

Questi limiti non annullano il valore dell’esperimento. I dati mostrano in modo chiaro che la ring dimension incide sul costo, che la somma è molto meno onerosa della moltiplicazione e che il bootstrapping è una fase eccezionalmente costosa. Sono proprio queste osservazioni a guidare le applicazioni dei capitoli successivi.

3.9 Conclusioni

I benchmark mostrano una tendenza chiara. Il packing permette di elaborare più valori in parallelo, ma dimensioni maggiori aumentano tempo e memoria.

Quando il sistema aggiunge moltiplicazioni, validazioni o training iterativo, il costo cambia sensibilmente. La profondità e le evaluation keys diventano centrali; il bootstrapping deve essere usato solo quando necessario. I benchmark costituiscono quindi il collegamento tra le caratteristiche teoriche degli schemi e le scelte architetturali dei due casi di studio.

Capitolo 4

Applicazioni di elaborazioni omomorfiche a scenari medicali

4.1 Introduzione

In questo capitolo si considera un'applicazione concreta: l'uso della regressione lineare in uno scenario medico. L'idea di fondo è semplice. Più strutture sanitarie o più soggetti che raccolgono dati clinici vogliono contribuire alla costruzione di un modello statistico, ma senza rinunciare alla protezione delle informazioni trattate.

Nel caso concreto considerato, la regressione lineare viene usata per stimare il numero di giorni di permanenza in ospedale di un paziente a partire da età, indice di comorbidità di Charlson, numero di procedure precedenti, pressione sistolica e diastolica e indice di massa corporea. Il capitolo mostra come questo modello venga inserito in un'architettura i cui attori principali sono:

- il *setup authority*, che crea il contesto crittografico e genera le chiavi del sistema;
- il *data provider*, che possiede i dati clinici originari;
- il *computation server*, che esegue il training del modello o le valutazioni numeriche;
- il *query actor*, che desidera interrogare il modello;
- il *decryptor*, che può coincidere con un soggetto istituzionale, ad esempio il SSN, e che detiene la capacità di decifrare i risultati.

La separazione tra computation server e decryptor è intenzionale. Le computazioni omomorfiche possono richiedere tempi elevati e possono quindi essere affidate anche a soggetti terzi specializzati, senza esporre in chiaro i dati sensibili o le query dei pazienti. Il decryptor resta invece il soggetto che possiede la chiave segreta e rende leggibile solo il risultato finale. A monte, il setup authority inizializza il sistema e distribuisce chiave pubblica, chiave segreta ed eventuali evaluation keys agli attori autorizzati.

Nel primo scenario il query actor invia query cifrate al computation server e riceve un risultato cifrato, poi decifrato dal decryptor. Nel secondo il computation server invia il polinomio cifrato al decryptor, mentre il query actor interroga in chiaro il servizio del soggetto fiduciario.

4.2 Richiami sulla regressione lineare

La regressione lineare serve a trovare una relazione semplice tra dati in ingresso e dato in uscita. In altre parole, si parte da esempi già osservati e si cerca una formula che descriva il loro andamento medio.

Il dataset contiene più osservazioni, una per ciascun paziente. Ogni osservazione comprende sei valori di ingresso: età, indice di comorbidità di Charlson, numero di procedure precedenti, pressione sistolica, pressione diastolica e indice di massa corporea. A questi sei valori corrisponde un unico valore da prevedere, cioè il numero di giorni di permanenza in ospedale.

Nel caso con più variabili in ingresso, il modello si scrive come

$$\hat{y} = w^T x + b = \sum_{j=1}^6 w_j x_j + b, \quad (4.1)$$

dove:

- w contiene i coefficienti del modello;
- x contiene i sei valori di ingresso relativi al paziente;
- b è il bias;
- $\hat{y}$ è il numero stimato di giorni di permanenza in ospedale.

La regressione adottata è lineare perché tutte le variabili x_j compaiono esclusivamente alla prima potenza: il modello non contiene termini quadratici, potenze superiori o interazioni tra feature. Questa scelta costituisce una semplificazione intenzionale. Variabili come la pressione sistolica e diastolica possono infatti avere una relazione più complessa e non lineare con la durata del ricovero; in un'applicazione clinica completa si potrebbero quindi considerare trasformazioni delle feature o modelli differenti. Nel presente caso si mantiene invece una funzione affine di primo grado, poiché lo scopo principale è dimostrare in modo semplice e leggibile l'applicazione delle tecniche omomorfiche.

4.2.1 Funzione di costo

Per capire se la retta trovata è buona oppure no, bisogna confrontare le previsioni del modello con i valori reali. Durante l'ottimizzazione si considera la metà dell'errore

quadratico medio:

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2. \quad (4.2)$$

Il fattore $1/2$ semplifica il calcolo del gradiente e non cambia i valori di w e b che minimizzano la funzione. La MSE completa, pari a $2J(w, b)$, viene usata per monitorare l'andamento del training. Se $J(w, b)$ è piccolo, il modello rappresenta bene le osservazioni disponibili.

4.2.2 Determinazione dei coefficienti

I coefficienti non vengono ricavati calcolando direttamente l'inversa di una matrice. Il modello usa invece la *discesa del gradiente* (*gradient descent*), un procedimento iterativo che parte da coefficienti iniziali molto semplici e li corregge poco alla volta nella direzione che riduce la funzione di costo.

Il training parte da pesi e bias nulli e, per un numero prefissato di epoche, calcola la previsione, misura l'errore e aggiorna i coefficienti usando un learning rate η .

In forma compatta, la previsione del modello è

$$\hat{y} = Xw + b\mathbf{1}_n, \quad (4.3)$$

dove $\mathbf{1}_n$ è un vettore di n elementi uguali a uno e indica che lo stesso bias viene aggiunto alla previsione di ogni paziente. Successivamente si calcola il vettore degli errori:

$$E = \hat{y} - y. \quad (4.4)$$

Se una previsione è troppo alta rispetto al valore reale, l'errore sarà positivo; se invece è troppo bassa, l'errore sarà negativo.

Una volta ottenuto l'errore, il modello aggiorna i pesi tramite

$$w \leftarrow w - \eta \frac{1}{n} X^T E \quad (4.5)$$

e aggiorna il bias tramite

$$b \leftarrow b - \eta \cdot \frac{1}{n} \sum_{i=1}^n E_i. \quad (4.6)$$

Se il modello sbaglia sistematicamente in una direzione, i coefficienti vengono spostati gradualmente nella direzione opposta. La MSE viene calcolata a ogni epoca per verificare il miglioramento.

4.2.3 Interpretazione in ambito medico

Nel caso considerato, il modello assume la forma

$$\hat{y} = w_1 \text{ age} + w_2 \text{ cci} + w_3 \text{ num_procedures} + w_4 \text{ systolic} + w_5 \text{ diastolic} + w_6 \text{ bmi} + b, \quad (4.7)$$

dove $\hat{y}$ è la permanenza stimata in ospedale, espressa in giorni. I coefficienti descrivono il contributo delle singole feature:

- w_1 è associato all'età del paziente, compresa nell'intervallo $[0, 100]$ anni;
- w_2 è associato al Charlson Comorbidity Index, compreso nell'intervallo $[0, 37]$;
- w_3 è associato al numero di procedure o interventi precedenti, compreso nell'intervallo $[0, 20]$;
- w_4 è associato alla pressione sistolica, compresa nell'intervallo $[80, 200]$ mmHg;
- w_5 è associato alla pressione diastolica, compresa nell'intervallo $[50, 120]$ mmHg;
- w_6 è associato all'indice di massa corporea, compreso indicativamente nell'intervallo $[15, 40]$;
- b rappresenta il bias del modello.

Questi intervalli descrivono i valori attesi delle variabili di ingresso, non i possibili valori dei coefficienti appresi. Inoltre, i coefficienti esprimono associazioni apprese dal dataset e non relazioni causali.

4.3 Regressione lineare ed elaborazione omomorfica

Una volta costruito il modello, il passaggio successivo consiste nel capire come usarlo nei due scenari considerati. La regressione lineare si presta bene a questo contesto perché, una volta noti i coefficienti, la previsione si ottiene con un calcolo molto semplice:

$$\hat{y} = w^T x + b = \sum_{j=1}^d w_j x_j + b. \quad (4.8)$$

In sostanza, bisogna fare prodotti e somme. Questo rende il modello adatto a un uso pratico anche in un'architettura protetta, perché la fase di inferenza non richiede logiche troppo complicate.

L'implementazione in FHE usa lo schema CKKS. Questa scelta è utile perché consente di rappresentare ed elaborare anche valori non interi, caratteristica importante in regressione lineare quando coefficienti, medie o risultati intermedi non sono interi. Inoltre,

nell'implementazione si ricorre al *bootstrapping* per riazzerare la profondità moltiplicativa del ciphertext, poiché durante la regressione lineare, in particolare nelle fasi iterative di calcolo, si accumulano molte moltiplicazioni omomorfiche. CKKS introduce approssimazioni numeriche e quindi piccoli errori di arrotondamento, ma nel contesto considerato tali differenze sono trascurabili ai fini del risultato finale.

Nel calcolo in chiaro si può pensare a x , w ed y come a vettori e matrici ordinarie. In FHE, invece, questi dati devono essere inseriti negli slot di un ciphertext. Per questo motivo l'implementazione non lavora direttamente con la matrice nella sua forma più intuitiva, ma la riorganizza in blocchi che possano essere sommati e ruotati in modo efficiente.

Nel presente contesto si può scomporre il problema in due fasi:

- (a) **training del modello**: il computation server riceve i dati dai provider e costruisce i coefficienti della regressione;
- (b) **inferenza o query**: un attore interroga il modello per ottenere una previsione su un nuovo input.

Il punto interessante è che nei due scenari cambia soprattutto la fase di query. Il modello matematico resta lo stesso, ma cambia chi vede i dati in chiaro, chi esegue la valutazione finale e chi possiede le chiavi necessarie per decifrare.

4.4 Scenario 1: query cifrata al computation server

Nel primo scenario, il data provider invia i dati al computation server, che esegue la regressione lineare e costruisce il polinomio del modello. Successivamente, il query actor non invia un input in chiaro, ma una query cifrata. Il computation server valuta il modello sul dato cifrato e restituisce un risultato anch'esso cifrato. Infine, il query actor si rivolge a un servizio di decrypt, che può essere incarnato dal SSN, per ottenere il valore in chiaro.

4.4.1 Schema logico

Lo schema può essere rappresentato come segue, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Data Provider}_i \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(\text{record di training}_i), \quad (4.9)$$

$$\text{Computation Server} : \text{training su dati cifrati e costruzione di } \text{Enc}_{pk}(w, b), \quad (4.10)$$

$$\text{Query Actor} \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(x_q), \quad (4.11)$$

$$\text{Computation Server} : \text{valutazione del modello su } \text{Enc}_{pk}(x_q), \quad (4.12)$$

$$\text{Computation Server} \longrightarrow \text{Query Actor} : \text{Enc}_{pk}(w^T x_q + b), \quad (4.13)$$

$$\text{Query Actor} \longrightarrow \text{Decryptor/SSN} : \text{Enc}_{pk}(w^T x_q + b), \quad (4.14)$$

$$\text{Decryptor/SSN} : \text{Dec}_{sk}(\text{Enc}_{pk}(w^T x_q + b)), \quad (4.15)$$

$$\text{Decryptor/SSN} \longrightarrow \text{Query Actor} : w^T x_q + b. \quad (4.16)$$

Il computation server addestra il modello e valuta il polinomio su query cifrate; il query actor vede il risultato in chiaro solo dopo l'intervento del decryptor.

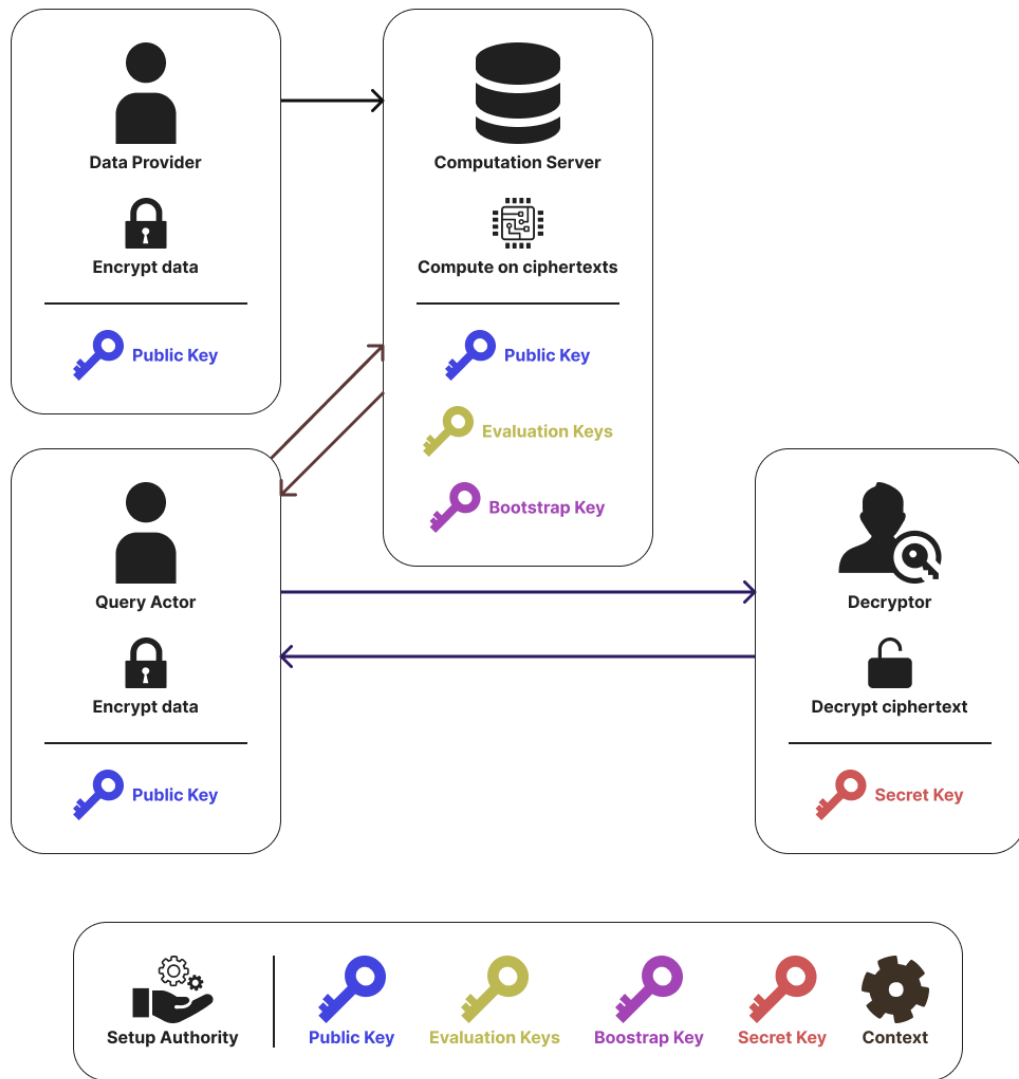


Figura 4.1: Schema dello scenario 1.

4.4.2 Distribuzione delle chiavi

In questo scenario si assume tipicamente la seguente ripartizione:

- il **setup authority** genera il contesto crittografico, la chiave pubblica pk , la chiave segreta sk ed eventuali evaluation keys, poi distribuisce tale materiale ai ruoli autorizzati;
- i **data provider** inviano al computation server soltanto dati cifrati con pk ;
- il **computation server** conosce la chiave pubblica pk e le eventuali evaluation keys necessarie alle operazioni omomorfiche;
- il **query actor** possiede o riceve la chiave pubblica pk per cifrare le query;

- il **decryptor/SSN** possiede la chiave segreta sk ;

La proprietà essenziale è che il computation server non dispone di sk , dunque non può vedere il contenuto delle query finali né dei risultati cifrati che produce.

Lo scenario protegge la query e separa chi calcola da chi decifra. Richiede però cifratura lato utente, gestione delle evaluation keys e l'intervento del decryptor, con maggiore complessità e latenza.

4.5 Scenario 2: il polinomio cifrato viene trasferito al decryptor

Nel secondo scenario, il data provider invia i dati al computation server, che esegue il training del modello. Una volta ottenuti i coefficienti della regressione, il computation server cifra il polinomio e lo trasferisce direttamente al decryptor, ad esempio il SSN. Da quel momento, il query actor non interagisce più con il computation server per la fase di inferenza, ma invia le proprie query in chiaro al soggetto fiduciario.

4.5.1 Schema logico

Lo schema può essere espresso come segue, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Data Provider}_i \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(\text{record di training}_i), \quad (4.17)$$

$$\text{Computation Server} : \text{training su dati cifrati e costruzione di } \text{Enc}_{pk}(w, b), \quad (4.18)$$

$$\text{Computation Server} \longrightarrow \text{Decryptor/SSN} : \text{Enc}_{pk}(w, b), \quad (4.19)$$

$$\text{Decryptor/SSN} : \text{Dec}_{sk}(\text{Enc}_{pk}(w, b)), \quad (4.20)$$

$$\text{Query Actor} \longrightarrow \text{Decryptor/SSN} : x_q \text{ in chiaro}, \quad (4.21)$$

$$\text{Decryptor/SSN} \longrightarrow \text{Query Actor} : w^T x_q + b. \quad (4.22)$$

Il query actor beneficia di un'interfaccia più semplice: non cifra nulla, ma si appoggia a un'entità trusted che custodisce il modello e produce le risposte.

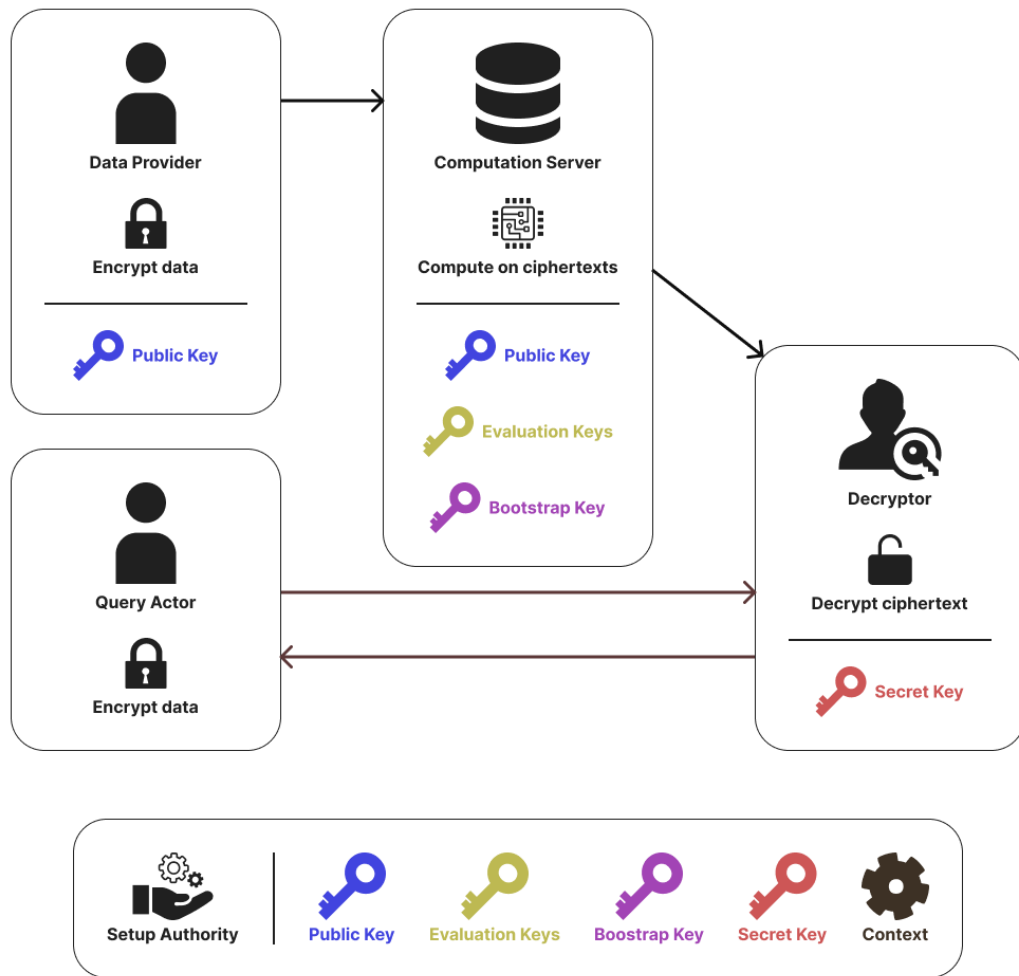


Figura 4.2: Schema dello scenario 2.

4.5.2 Distribuzione delle chiavi

La distribuzione delle chiavi è più centralizzata:

- il **setup authority** genera il contesto crittografico, la chiave pubblica pk , la chiave segreta sk ed eventuali evaluation keys, poi le distribuisce agli attori autorizzati;
- i **data provider** inviano al computation server soltanto dati cifrati con pk ;
- il **computation server** conosce la chiave pubblica pk e le eventuali evaluation keys necessarie alle operazioni omomorfiche;
- il **query actor** non ha bisogno di utilizzare direttamente chiavi crittografiche nella fase di interrogazione;
- il **decryptor/SSN** possiede la chiave segreta sk ;

Questa architettura è coerente con contesti in cui il decryptor rappresenta un ente istituzionale già riconosciuto come fiduciario, in grado di custodire chiavi, modelli e risultati.

Lo scenario semplifica l'interrogazione e riduce la latenza, ma espone l'input al decryptor e concentra informazioni e fiducia presso tale ente. È quindi adatto quando la fiducia nel soggetto istituzionale è alta.

4.6 Confronto tra le due architetture

Le due architetture risolvono lo stesso problema, ma ottimizzano aspetti diversi. Una sintesi comparativa può essere espressa nella Tabella 4.1.

Aspetto	Scenario 1	Scenario 2
Protezione delle query	Alta: il computation server riceve query cifrate	Più bassa: il query actor invia la query in chiaro al decryptor/SSN
Ruolo del computation server	Addestra il modello e valuta query cifrate	Addestra il modello e invia il polinomio cifrato al decryptor
Ruolo del decryptor	Decifra il risultato finale	Custodisce il modello e risponde alle query
Chiave segreta	Presso il decryptor/SSN	Presso il decryptor/SSN
Chiave pubblica	Usata da query actor e computation server	Usata da query actor e computation server
Complessità per l'utente finale	Maggiore: deve cifrare la query	Minore: usa un'interfaccia in chiaro
Latenza complessiva	Più elevata	Più contenuta, ma con costo di inferenza a carico del decryptor/SSN

Tabella 4.1: Confronto tra i due scenari di impiego della regressione lineare in ambito omomorfo-medico.

Da questa tabella emerge un trade-off classico. Se si vuole proteggere fortemente la riservatezza dell'input di inferenza, lo scenario 1 è generalmente preferibile. Se invece si desidera un servizio più semplice, controllato da un ente sanitario centrale e più immediato da usare, lo scenario 2 può risultare più naturale. Va però ricordato che, nello scenario 2, il training resta a carico del computation server mentre l'inferenza viene eseguita dal decryptor, cioè dal SSN: anche questa fase ha quindi un costo computazionale e operativo che deve essere sostenuto dall'attore ultimo invece del server computazionale che però essendo in chiaro risultat molto meno dispendioso del eseguire una decifrazione.

4.7 Esempio applicativo: stima dei giorni di ricovero

Il dataset sanitario contiene, per ogni osservazione, le sei caratteristiche del paziente e il numero di giorni di permanenza in ospedale da apprendere. Alcune osservazioni sono:

Età	CCI	Proc.	Sist.	Diast.	BMI	Giorni
37	2	12	148	106	22.5	4
77	1	7	105	106	38.0	3
83	2	17	82	64	29.2	5

Il modello deve trovare i sei pesi e il bias che minimizzano l'errore tra i giorni stimati e quelli presenti nell'ultima colonna.

4.7.1 Come parte il training

Il training parte da coefficienti iniziali nulli:

$$w_1 = \dots = w_6 = 0, \quad b = 0. \quad (4.23)$$

Con questa scelta, la previsione iniziale per ogni paziente è zero. Il computation server confronta tali previsioni con i giorni di permanenza osservati e usa gli errori per correggere pesi e bias. Il procedimento viene ripetuto epoca dopo epoca: i parametri non sono ottenuti in un singolo passaggio, ma attraverso un miglioramento progressivo del modello.

4.7.2 Quale modello viene recuperato

Il generatore costruisce una durata attesa a partire dalle caratteristiche del paziente. Età, CCI e numero di procedure forniscono un contributo di base; valori pressori o BMI esterni agli intervalli considerati normali aggiungono un ulteriore contributo.

Il dataset non segue quindi una relazione lineare esatta. La regressione apprende una funzione affine che approssima l'andamento complessivo:

$$\widehat{\text{days_in_hospital}} = w^T x + b. \quad (4.24)$$

L'uscita va interpretata come una stima della permanenza e non come un valore clinico certo.

4.7.3 Verifica su un nuovo dato

Una query contiene le sei caratteristiche di un nuovo paziente, per esempio:

$$x_q = (50, 4, 20, 128, 96, 26.7). \quad (4.25)$$

Il modello calcola $\hat{y} = w^T x_q + b$ e restituisce il numero stimato di giorni di permanenza in ospedale. Una volta ottenuti i coefficienti, la query richiede soltanto prodotti e somme e si presta quindi alla valutazione omomorfica.

4.7.4 Lettura dell'esempio nei due scenari

Nello **scenario 1**, il query actor cifra il vettore clinico x_q e lo invia al computation server. Il server applica il modello al dato cifrato e produce

$$\text{Enc}_{pk}(w^T x_q + b), \quad (4.26)$$

cioè la stima cifrata dei giorni di ricovero. Il risultato viene poi decifrato dal SSN.

Nello **scenario 2**, il computation server ha già trasmesso il polinomio cifrato al decryptor. Il SSN custodisce i coefficienti e riceve in chiaro il vettore clinico del paziente, sul quale calcola $w^T x_q + b$. Il valore matematico finale è lo stesso; cambia il punto del sistema in cui i dati del paziente vengono esposti in chiaro.

4.8 Aspetti operativi e limiti

La cifratura omomorfica consente di analizzare dati distribuiti tra strutture sanitarie senza condividerli in forma leggibile. La semplicità della regressione lineare la rende adatta a progetti pilota nei quali la trasparenza del modello è importante.

Restano limiti concreti: CKKS introduce piccole approssimazioni numeriche, il costo computazionale supera quello dell'elaborazione in chiaro e occorre gestire chiavi, audit, responsabilità e validazione clinica.

4.9 Conclusioni

La regressione lineare rappresenta un caso di studio efficace per mostrare come la cifratura omomorfica possa essere applicata a scenari medicali concreti. Il computation server costruisce un polinomio di primo grado, o più in generale una funzione affine multivariata, a partire da dati clinici raccolti dai data provider. Successivamente tale modello può essere interrogato secondo architetture differenti, che riflettono scelte diverse in termini di privacy, fiducia e semplicità operativa.

Lo scenario 1 privilegia la protezione della query e riduce l'esposizione del dato di input al computation server, ma richiede una pipeline più articolata. Lo scenario 2, invece, centralizza la fiducia nel decryptor, semplifica l'interrogazione e può adattarsi bene a una governance sanitaria istituzionale, pur esponendo in chiaro la query al soggetto fiduciario.

Capitolo 5

Protocollo di aggregazione simil prio3 utilizzando tecniche omomorfiche

5.1 Introduzione

Prio3 VDAF è, in modo molto semplice, un protocollo pensato per raccogliere dati da più client e calcolarne un risultato aggregato senza esporre il contributo individuale di ciascuno [6]. L'idea generale è che ogni client non invii direttamente il proprio valore in chiaro a un unico soggetto, ma produca una rappresentazione adatta a essere elaborata da più attori del protocollo. In questo modo, il sistema finale può recuperare il risultato complessivo senza ricostruire facilmente il dato del singolo partecipante.

L'obiettivo di questo capitolo è allora il seguente: replicare in ambiente FHE la forma generale di un protocollo di aggregazione ispirato a Prio3, mantenendo l'idea di fondo di raccolta privata e di validazione dei report, ma semplificando l'architettura. La costruzione che introduciamo, chiamata `fhe-vdaf`, usa lo schema omomorfo BGV: i report vengono cifrati dai client, inviati a un unico aggregatore, validati sotto cifratura e poi sommati, in modo che l'attore finale possa apprendere soltanto il risultato aggregato.

La scelta di BGV non è casuale. In questo contesto i report sono valori interi e tutta la logica di validazione viene espressa tramite aritmetica modulare. BGV è quindi naturale perché lavora direttamente su interi modulo p . Questo significa che somme e moltiplicazioni avvengono nello stesso ambiente algebrico in cui vogliamo controllare i report, compreso il comportamento di overflow, che viene assorbito in modo coerente dalla riduzione modulo p .

5.2 Idea generale di `fhe-vdaf`

La struttura di `fhe-vdaf` può essere descritta in modo molto semplice:

- (a) il client vuole inviare un valore numerico;

- (b) lo converte in una rappresentazione binaria a slot;
- (c) cifra tale rappresentazione con la chiave pubblica del sistema;
- (d) invia il ciphertext all'aggregatore;
- (e) l'aggregatore valida omomorficamente il report;
- (f) se il report è valido, lo aggiunge alla somma aggregata;
- (g) il collector finale decifra solo il totale.

In pratica, tutti i report cifrati arrivano allo stesso aggregatore. La crittografia di base resta la stessa, ma cambia il modo in cui il sistema distribuisce i compiti e la fiducia tra gli attori.

5.3 Ruoli architetturali e gestione delle chiavi

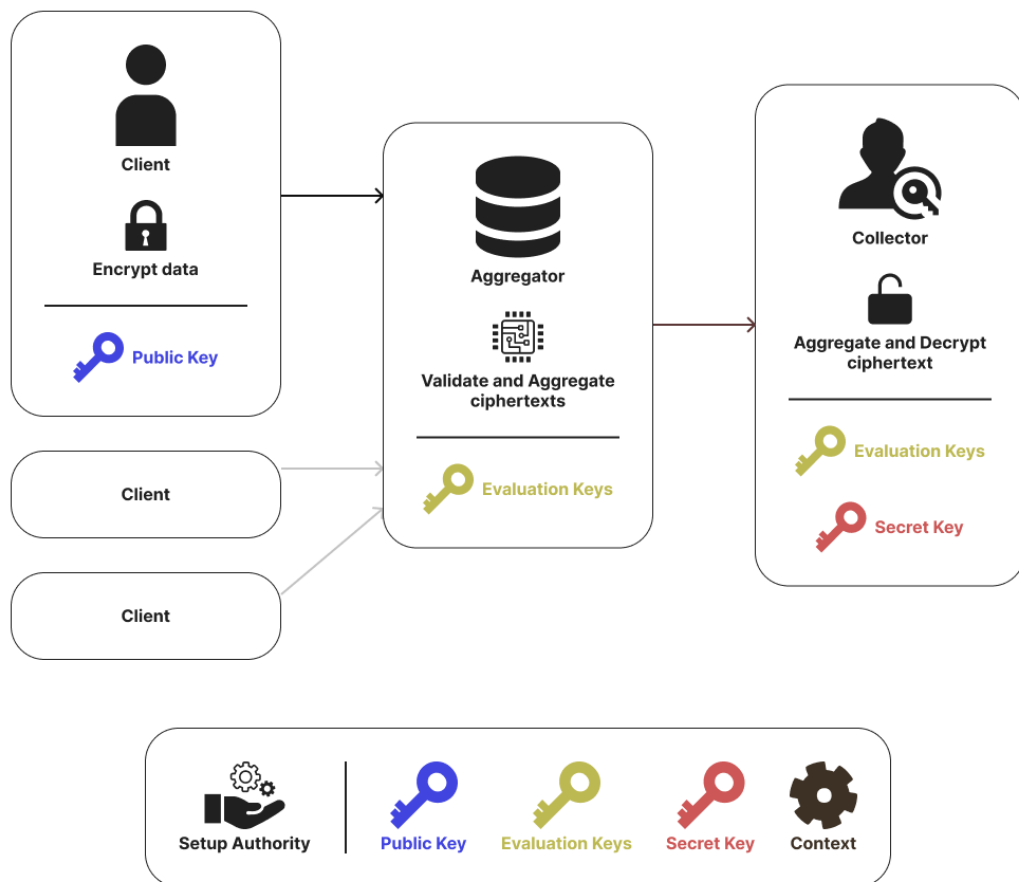


Figura 5.1: Schema di fhe-vdaf.

5.3.1 Attori del sistema

Nel caso di **fhe-vdaf** si possono distinguere quattro attori principali:

- il *setup authority*, che crea il contesto crittografico e le chiavi;
- i *client*;
- l'*aggregator*, che riceve i report cifrati;
- il *collector*, che decifra il risultato finale.

5.3.2 Chi possiede quali chiavi

Dal punto di vista della gestione delle chiavi, **fhe-vdaf** conserva la distinzione tra cifratura, valutazione omomorfica e decifratura finale.

La distribuzione tipica è la seguente:

- i **client** possiedono il contesto pubblico e la **chiave pubblica** pk , necessaria per cifrare i report;
- l'**aggregator** possiede il contesto crittografico e le **evaluation keys**, cioè le chiavi ausiliarie per eseguire somme, rotazioni e moltiplicazioni omomorfe;
- il **collector** possiede la **chiave segreta** sk e può decifrare il risultato finale;
- il **setup authority** genera inizialmente pk , sk ed evaluation keys, e poi le distribuisce ai ruoli autorizzati.

Questa separazione è importante. L'aggregatore può elaborare i dati cifrati ma, in condizioni normali, non può decifrarli. Il collector può decifrare, ma non riceve i singoli report in chiaro durante la fase di raccolta.

5.3.3 Conseguenze di sicurezza

L'aspetto più delicato di **fhe-vdaf** è la concentrazione dello schema presso l'aggregatore. Questo comporta che:

- tutti i ciphertext passano per lo stesso nodo;
- tutta la validazione viene eseguita nello stesso punto;
- non c'è ridondanza fiduciaria tra aggregatori distinti;
- un'eventuale collusione tra aggregatore e soggetto che controlla la chiave segreta comprometterebbe la protezione dei report individuali.

Per questo motivo, **fhe-vdaf** è molto semplice da capire, ma meno forte di una soluzione con più aggregatori indipendenti.

5.4 Architettura complessiva in uno scenario di dati aggregati

In `fhe-vdaf`, tutti i report vengono cifrati e inviati all'aggregatore. L'aggregatore non deve conoscere il contenuto di ciascun numero, ma deve poter:

- verificare che ogni report sia ben formato;
- scartare i report invalidi;
- sommare i report validi.

Infine, il collector riceve la somma cifrata finale e la decifra, ottenendo solo il totale aggregato.

5.4.1 Schema dei messaggi

Lo schema architetturale può essere rappresentato così, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Client}_i \longrightarrow \text{Aggregator} : \text{Enc}_{pk}(\text{report}_i), \quad (5.1)$$

$$\text{Aggregator} : \text{validate-or-zero}(\text{Enc}_{pk}(\text{report}_i)), \quad (5.2)$$

$$\text{Aggregator} : \sum_i \text{Enc}_{pk}(\text{report}_i), \quad (5.3)$$

$$\text{Aggregator} \longrightarrow \text{Collector} : \text{Enc}_{pk}(\text{totale}), \quad (5.4)$$

$$\text{Collector} : \text{Dec}_{sk}(\text{Enc}_{pk}(\text{totale})). \quad (5.5)$$

5.5 Conversione dell'input in binario

5.5.1 Perché usare una rappresentazione binaria

Il cuore operativo del protocollo è la trasformazione del dato numerico in una sequenza di slot binari. Questa scelta non è un dettaglio marginale: è proprio ciò che consente all'aggregatore di verificare, anche sotto cifratura, che il report abbia una forma lecita.

La scomposizione in bit permette di ridurre il controllo a una proprietà locale:

$$x \in \{0, 1\}. \quad (5.6)$$

5.5.2 Vincolo sul valore massimo

Nel prototipo considerato, il valore massimo consentito è

$$\text{MAX_SIZE} = 100. \quad (5.7)$$

Per rappresentare correttamente tutti i valori interi compresi tra 0 e `MAX_SIZE`, il numero minimo di bit richiesto si calcola con

$$\text{MAX_BITS} = \lfloor \log_2(\text{MAX_SIZE}) \rfloor + 1, \quad (5.8)$$

valida per `MAX_SIZE` > 0.

Nel nostro caso:

$$\lfloor \log_2(100) \rfloor + 1 = \lfloor 6.64 \dots \rfloor + 1 = 6 + 1 = 7. \quad (5.9)$$

Quindi

$$\text{MAX_BITS} = 7. \quad (5.10)$$

La codifica non è una rappresentazione binaria standard perfettamente uniforme. I primi sei slot usano i pesi classici

$$1, 2, 4, 8, 16, 32, \quad (5.11)$$

mentre l'ultimo slot usa il peso

$$37. \quad (5.12)$$

La ragione è che si vuole far corrispondere il vettore di tutti uno al valore massimo 100:

$$1 + 2 + 4 + 8 + 16 + 32 + 37 = 100. \quad (5.13)$$

In questo modo, tutti i valori ammessi tra 0 e 100 possono essere rappresentati con sette slot binari.

La logica di codifica seguita dal codice è la seguente. Si considerano prima i primi sei bit, che possono rappresentare direttamente tutti i valori compresi tra 0 e

$$\text{LOW_BITS_MAX_VALUE} = 2^6 - 1 = 63. \quad (5.14)$$

Se il valore da codificare soddisfa

$$v \leq 63, \quad (5.15)$$

allora viene codificato normalmente nei primi sei bit e l'ultimo bit resta a zero.

Se invece

$$v > 63, \quad (5.16)$$

allora il codice:

- (a) pone l'ultimo bit uguale a 1;

- (b) sottrae il suo peso, cioè 37, dal valore originale;
- (c) codifica il valore rimanente nei primi sei bit.

Il valore finale decodificato sarà quindi sempre

$$v = v_{\text{low}} + 37 \cdot b_7. \quad (5.17)$$

5.5.3 Esempi di codifica

Un esempio più interessante è un valore maggiore di 63, ad esempio 64. In questo caso il codice accende l'ultimo bit e poi codifica il resto:

$$64 - 37 = 27. \quad (5.18)$$

Poiché

$$27 = 1 + 2 + 8 + 16, \quad (5.19)$$

si ottiene

$$64 \mapsto [1, 1, 0, 1, 1, 0, 1]. \quad (5.20)$$

Infine, il valore massimo:

$$100 \mapsto [1, 1, 1, 1, 1, 1, 1]. \quad (5.21)$$

Infatti, in questo caso:

$$100 - 37 = 63, \quad (5.22)$$

e 63 viene rappresentato dai primi sei bit tutti a uno:

$$63 = 1 + 2 + 4 + 8 + 16 + 32. \quad (5.23)$$

5.6 Validazione omomorfica dell'input

5.6.1 Controllo che ogni slot sia un bit

Una volta ricevuto un ciphertext, l'aggregatore deve verificare che ogni slot codifichi davvero un bit, cioè un valore 0 oppure 1.

Il controllo fondamentale usa il polinomio elementare

$$f(x) = x(x - 1). \quad (5.24)$$

Quindi:

- se uno slot contiene 0 o 1, il controllo restituisce zero;
- se contiene un altro valore, compare un errore non nullo.

Se il ciphertext contiene gli slot $x_1, \dots, x_{\text{MAX_BITS}}$, l'aggregatore calcola per ogni slot

$$e_i = x_i(x_i - 1) \quad (5.25)$$

e poi somma tutti gli errori:

$$E = \sum_{i=1}^{\text{MAX_BITS}} e_i. \quad (5.26)$$

Se il report è ben formato, allora

$$E = 0. \quad (5.27)$$

Se invece almeno uno slot non è binario, allora

$$E \neq 0. \quad (5.28)$$

5.6.2 Da errore a flag di validità

Il valore E è ancora cifrato. L'aggregatore non può leggerlo direttamente, ma può trasformarlo in un flag di errore sfruttando l'aritmetica del campo finito usato dallo schema BGV.

Nel prototipo viene usato il modulo primo

$$p = 786433. \quad (5.29)$$

Per il piccolo teorema di Fermat, per ogni elemento non nullo vale

$$E^{p-1} = 1 \pmod{p}, \quad (5.30)$$

mentre se $E = 0$ allora

$$E^{p-1} = 0. \quad (5.31)$$

Poiché

$$p - 1 = 786432 = 3 \cdot 2^{18}, \quad (5.32)$$

l'aggregatore può costruire il flag di errore come

$$\text{is_error} = E^{786432}. \quad (5.33)$$

Il significato è:

- $\text{is_error} = 0$ se il report è valido;
- $\text{is_error} = 1$ se il report è invalido.

Da qui si ottiene il flag opposto:

$$\text{is_ok} = 1 - \text{is_error}. \quad (5.34)$$

Quindi:

- $\text{is_ok} = 1$ per i report validi;
- $\text{is_ok} = 0$ per i report invalidi.

5.6.3 Validate-or-zero

A questo punto l'aggregatore moltiplica il report cifrato per il flag di validità:

$$\text{report}_{\text{checked}} = \text{report} \cdot \text{is_ok}. \quad (5.35)$$

Il risultato è molto intuitivo:

- se il report è valido, viene moltiplicato per 1 e resta invariato;
- se il report è invalido, viene moltiplicato per 0 e diventa il vettore nullo.

Questa tecnica è spesso descritta come *validate-or-zero*. In un prototipo dimostrativo è utile perché consente di continuare l'aggregazione senza interrompere l'esecuzione per ogni input scorretto.

5.7 Aggregazione dei report validi

Una volta validati tutti i report, l'aggregatore li somma omomorficamente. Se i report validi sono $c_1, \dots, c_m$, la somma aggregata è

$$C_{\text{sum}} = c_1 + c_2 + \dots + c_m. \quad (5.36)$$

Poiché ogni ciphertext contiene i sette slot della codifica binaria, il risultato della somma è un nuovo vettore cifrato i cui slot rappresentano la somma posizione per posizione dei bit validi ricevuti.

Il collector riceve questo ciphertext finale e lo decifra. Dopo la decifratura, non ottiene direttamente il totale come singolo intero, ma i sette slot aggregati:

$$[s_1, s_2, s_3, s_4, s_5, s_6, s_7]. \quad (5.37)$$

Il totale finale viene quindi ricostruito come

$$\text{totale} = 1 \cdot s_1 + 2 \cdot s_2 + 4 \cdot s_3 + 8 \cdot s_4 + 16 \cdot s_5 + 32 \cdot s_6 + 37 \cdot s_7. \quad (5.38)$$

Questo passaggio è analogo alla funzione di *decode* del valore usata nel progetto.

5.8 Esempio completo su dati aggregati

5.8.1 Codifica dei valori

La codifica binaria a sette slot produce:

$$51 \mapsto [1, 1, 0, 0, 1, 1, 0], \quad (5.39)$$

$$7 \mapsto [1, 1, 1, 0, 0, 0, 0], \quad (5.40)$$

$$4 \mapsto [0, 0, 1, 0, 0, 0, 0], \quad (5.41)$$

$$49 \mapsto [1, 0, 0, 0, 1, 1, 0], \quad (5.42)$$

$$13 \mapsto [1, 0, 1, 1, 0, 0, 0]. \quad (5.43)$$

Ogni vettore viene cifrato con la chiave pubblica pk e inviato all'aggregatore.

5.8.2 Somma per slot

Dopo la decifratura, il collector ottiene la somma per slot:

$$[4, 2, 3, 1, 2, 2, 0]. \quad (5.44)$$

5.8.3 Decodifica finale

Il totale finale si ricostruisce come:

$$\text{totale} = 1 \cdot 4 + 2 \cdot 2 + 4 \cdot 3 + 8 \cdot 1 + 16 \cdot 2 + 32 \cdot 2 + 37 \cdot 0 \quad (5.45)$$

$$= 4 + 4 + 12 + 8 + 32 + 64 + 0 \quad (5.46)$$

$$= 124. \quad (5.47)$$

Il collector apprende dunque solo il totale

$$124, \quad (5.48)$$

senza aver partecipato alla validazione dei singoli report e senza aver ricevuto i report individuali in chiaro durante la fase di raccolta.

5.9 Punti di forza e limiti architetturali

L'architettura adottata presenta una serie di vantaggi pratici, ma anche limiti precisi che vanno esplicitati con chiarezza.

5.9.1 Vantaggi

I vantaggi principali di **fhe-vdaf** sono:

- semplicità architetturale;
- minore complessità di orchestrazione;
- minore frammentazione degli artifact e dei flussi;
- facilità di integrazione in sistemi centralizzati.

5.9.2 Limiti

I principali limiti sono:

- assenza di distribuzione fiduciaria tra aggregatori distinti;
- concentrazione di tutti i ciphertext presso lo stesso nodo;
- maggiore esposizione in caso di compromissione dell'aggregatore;
- minore vicinanza alle costruzioni VDAF più robuste.

5.10 Conclusioni

fhe-vdaf può essere visto come una forma molto semplice di raccolta privata di conteggi basata su cifratura omomorfica e validazione VDAF-like. Il meccanismo centrale è quello della codifica binaria del valore, del controllo elementare di validità e dell'aggregazione cifrata presso un unico nodo.

Il capitolo ha evidenziato tre aspetti principali. Il primo riguarda l'architettura: client, aggregatore unico e collector con chiavi chiaramente separate. Il secondo riguarda la codifica dell'input, che trasforma ogni valore ammesso in un vettore di bit pesati. Il terzo riguarda la validazione, basata su controlli polinomiali elementari ma molto efficaci per riconoscere valori non binari.

In uno scenario reale, questa architettura può essere utile come base sperimentale o come soluzione centralizzata per aggregazioni relativamente semplici. Resta però importante ricordare che la semplicità di **fhe-vdaf** corrisponde anche a una minore distribuzione del trust. Di conseguenza, il suo impiego va valutato nel quadro più ampio della

governance del sistema, della gestione delle chiavi e delle garanzie organizzative offerte dall'infrastruttura che lo ospita.

Capitolo 6

Conclusioni

La cifratura omomorfica permette di ripensare il rapporto tra dati e calcolo: il soggetto che elabora non deve necessariamente vedere ciò che elabora. Questa possibilità apre scenari utili, ma introduce costi e vincoli che devono essere misurati e compresi.

La tesi ha seguito un percorso intenzionalmente graduale. La prima parte ha chiarito il ruolo e le differenze operative tra BFV, BGV e CKKS. La seconda ha mostrato, tramite benchmark, che tempo, memoria, profondità moltiplicativa e gestione delle chiavi di valutazione incidono in modo concreto sulla fattibilità di un'applicazione.

Su questa base sono stati analizzati due casi di studio. Il primo ha riguardato la regressione lineare in ambito medico, dove CKKS consente di trattare quantità numeriche approssimate mantenendo il dato protetto durante la fase di calcolo. Il secondo ha proposto **fhe-vdaf**, una costruzione ispirata alle VDAF che usa BGV per validare e aggregare report cifrati senza esporre direttamente i contributi individuali.

L'insieme dei risultati mostra che la privacy non dipende soltanto dalla presenza di uno schema crittografico avanzato. Dipende anche dal tipo di dato da rappresentare, dai parametri scelti, dal costo delle operazioni supportate e dall'architettura che distribuisce chiavi, ruoli e fiducia tra gli attori del sistema.

In sintesi, la cifratura omomorfica non elimina i compromessi del calcolo sicuro, ma offre strumenti concreti per costruire sistemi in cui l'elaborazione dei dati sensibili sia più controllata. In questo senso, teoria, benchmark e applicazioni non restano parti separate, ma descrivono fasi successive dello stesso problema ingegneristico.

Bibliografia

- [1] OpenFHE Development Team, *OpenFHE Documentation: Examples and Supported Schemes*. https://openfhe-development.readthedocs.io/en/latest/sphinx_rsts/intro/quickstart.html.
- [2] J. H. Cheon, A. Kim, M. Kim e Y. Song, *Homomorphic Encryption for Arithmetic of Approximate Numbers*, Cryptology ePrint Archive, Paper 2016/421, 2016. <https://eprint.iacr.org/2016/421>.
- [3] Z. Brakerski, C. Gentry e V. Vaikuntanathan, *Fully Homomorphic Encryption without Bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. <https://eprint.iacr.org/2011/277>.
- [4] J. Fan e F. Vercauteren, *Somewhat Practical Fully Homomorphic Encryption*, Cryptology ePrint Archive, Paper 2012/144, 2012. <https://eprint.iacr.org/2012/144>.
- [5] D. I. Parvu, *FHE Benchmark Suite Repository*. <https://github.com/kratess/FHE/tree/main/benchmark>.
- [6] R. Barnes, D. Cook, C. Patton e P. Schoppmann, *Verifiable Distributed Aggregation Functions*, Internet-Draft draft-irtf-cfrg-vdaf-19, Internet Engineering Task Force, Work in Progress, 14 aprile 2026. <https://datatracker.ietf.org/doc/draft-irtf-cfrg-vdaf/19/>.